

JobTrack

Understand the app, follow how information moves through it, and find your way around the code.

Your job search, in one organized place.

JobTrack helps you remember where you applied, what happened next, and when to follow up. You can also keep PDF resume copies with your account.

What this guide covers

Use the product

Learn the application, status, reminder, resume, and dashboard workflows.

Understand the system

See how the browser, API server, and database work together.

Explore the code

Find the files responsible for screens, API routes, data, and tests.

Run it locally

Start the project, create an account, and try the features with safe sample data.

Important: JobTrack stores resumes, but it does not score them. The separate ATS scoring feature has been removed from this project.

Project stack: React + TypeScript · FastAPI · SQLAlchemy · MySQL · Docker Compose

This guide describes the current JobTrack codebase. Use sample data when exploring; do not upload a real resume to a public demo.

What is JobTrack?

When people apply to many jobs, details quickly get scattered across browser tabs, email, and notes. JobTrack gives each opportunity a record that stays organized.

Application

A record for one role at one company. It can include a link, location, application date, notes, and job description.

Status

A simple label for the current stage: wishlist, applied, online assessment, interview, offer, rejected, or withdrawn.

Status history

A timeline of changes. Updating an application records what its old and new statuses were.

Reminder

A dated follow-up attached to an application, such as "Email recruiter next Tuesday."

A typical job-search loop

1. **Save an opportunity.** Add its company, role, and useful details.
2. **Record progress.** Change the status when you apply, get an interview, or hear back.
3. **Plan the next action.** Add a reminder so a follow-up does not get forgotten.
4. **Review patterns.** Use the dashboard to see activity and response trends.

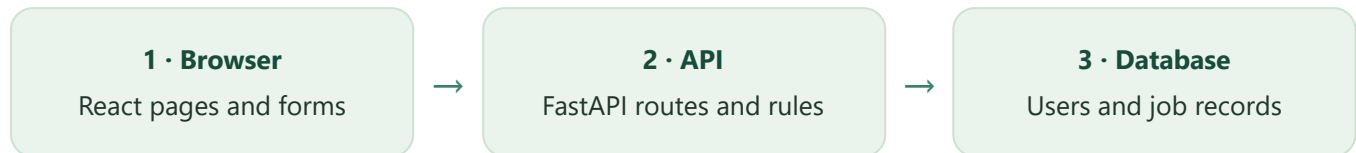
What a "full-stack" app means

A full-stack application has the parts a user sees and the parts that store and process their data. The **frontend** is the website. The **backend** is the API that applies rules and talks to the database. The **database** keeps information after you close the browser.

Simple analogy: The frontend is a service counter, the API is the staff member checking the request, and the database is the filing system. The counter does not access private files directly; the API checks who is asking and what they are allowed to do.

Architecture at a glance

When you click a button, the browser sends a request to the API. The API checks the request and your account, reads or changes records in the database, then sends a response back for the browser to display.



Example: creating an application

1. You fill in the form in the React frontend.
2. The frontend sends a JSON `POST /api/applications` request with your sign-in token.
3. FastAPI checks the request shape and identifies your account from the token.
4. SQLAlchemy saves the application and its initial status-history entry.
5. The API returns the new application; the frontend refreshes the list.

Why there are three services in local Compose

Service	Job	Typical local address
Frontend	Displays the screens and sends API requests.	<code>localhost:3000</code>
API	Validates requests, checks authentication, runs product rules.	<code>localhost:8000</code>
MySQL	Persists accounts, applications, reminders, and resume bytes.	Internal Compose network

Key words you will see in the code

Word	Plain-language meaning
HTTP request	A message asking a server to do something or return information.
Endpoint / route	A named API address, for example <code>GET /api/stats</code> .
JSON	A common text format for sending structured data between frontend and backend.
ORM	A tool (SQLAlchemy here) that maps Python objects to database rows.
Docker Compose	A tool that starts the app's containers together from one configuration file.

What can you do in the app?

Sign up and sign in

Create an account with your email and password. The backend hashes the password before saving it. When you sign in, it returns a time-limited bearer token; the frontend sends that token with later requests.

Add and track a job

Open **Applications**, add a company and role, and fill in any other details you know. The job description is optional for tracking. New applications start in the wishlist status. Open a record to view its details, add notes, and update its status.

Use history and reminders

Changing a status creates a history event instead of erasing the old state. Add a reminder with a time zone and JobTrack normalizes the stored date to UTC. The dashboard can highlight reminders that are due.

Keep resume copies

The **Resumes** page lets you upload, label, list, and delete PDF copies. The backend checks the PDF signature and filename, limits each file to 2 MB, and allows up to five per account. File bytes stay in the database. The current project does **not** compare or score resumes.

Read the dashboard

The stats endpoint summarizes statuses, applications per week, response rate, and time to first response. These are calculated from the dates and history entered, so incomplete or backfilled information can affect the summary.

Ownership matters: Routes look up applications and resumes through the signed-in account. One user should not be able to read, change, or delete another user's records.

Explore API docs

When the API is running, open <http://localhost:8000/docs>. This interactive page lists endpoints, expected inputs, and response shapes. Try it with a local test account, not private or production data.

Where does the server code live?

Start in `backend/app/`. A request enters through a router, uses schemas to validate input, checks identity with security helpers, and reads/writes models through the database session.

File or folder	What it does	Good first question
<code>main.py</code>	Creates FastAPI, adds CORS, registers routes, exposes health.	Which routers are part of the app?
<code>config.py</code>	Loads required environment settings and rejects invalid secrets.	Which settings must exist at startup?
<code>db.py</code>	Builds the database engine and creates per-request sessions.	How does a route get its database session?
<code>models.py</code>	Defines User, Application, StatusHistory, Reminder, and Resume tables.	Which data belongs to which account?
<code>schemas.py</code>	Defines validated request and response shapes.	Which fields may the frontend send?
<code>security.py</code>	Hashes/verifies passwords and creates/reads JWTs.	How is the current user identified?
<code>routers/auth.py</code>	Signup, login, profile, and account deletion.	How does a session begin or end?
<code>routers/applications.py</code>	Application CRUD and status-history routes.	Where is ownership checked?
<code>routers/reminders.py</code>	Creates, lists, and completes reminders.	How are due reminders filtered?
<code>routers/resumes.py</code>	Validates, stores, lists, and deletes uploaded PDFs.	What are the file-size/type limits?
<code>routers/stats.py</code> + <code>services/stats_service.py</code>	Exposes dashboard statistics and performs calculations.	How are responses and weeks counted?

Tests describe expected behavior

`backend/tests/` has focused tests for authentication, applications, history, reminders, resume file validation, stats, and health. Tests use a local SQLite database so you do not need production data to verify these rules.

Backend request pattern

Router endpoint → Pydantic schema → current-user dependency → SQLAlchemy model/session → response schema

The exact flow varies by route, but this sequence is a useful map when reading a feature.

Where does the website code live?

Start in `frontend/src/`. Pages compose reusable components. The API client sends network requests, and TanStack Query keeps server data in sync with the screens.

File or folder	What it does
<code>main.tsx</code>	Mounts the React app in the browser document.
<code>App.tsx</code>	Defines routes such as login, dashboard, applications, resumes, and settings.
<code>pages/</code>	One screen per workflow: dashboard, list/detail, authentication, resume management.
<code>components/</code>	Reusable pieces such as forms, navigation, error states, and notifications.
<code>api/client.ts</code>	Centralizes API calls, bearer token handling, and API errors.
<code>api/generated.ts</code>	Types generated from the backend OpenAPI schema; regenerate after API changes.
<code>api/format.ts</code>	Shared date and status display helpers.
<code>styles.css</code>	Global visual styles and layout.
<code>test/</code> and <code>*.test.tsx</code>	Frontend test setup and automated component/client behavior checks.

Three useful HTTP verbs

Verb	Meaning	Example
GET	Read data.	List your applications.
POST	Create something or start an action.	Create an application.
PATCH	Change part of an existing record.	Update a status or mark a reminder done.
DELETE	Remove a record.	Delete an application or resume.

Reading a frontend feature end to end

1. Find the page in `pages/` that shows the feature.
2. Find the `api.*` function it calls in `api/client.ts`.
3. Find the matching route in `backend/app/routers/`.
4. Check the schema, model, and test to understand validation and behavior.

Start JobTrack on your computer

The simplest full-stack setup uses Docker Compose. It starts MySQL, the FastAPI server, and the production-style frontend together.

1. Install and start Docker Desktop.
2. In the project root, copy `.env.example` to a new file named `.env`.
3. Set local database passwords and a random `JWT_SECRET` of at least 32 bytes.
4. Open PowerShell in the project root and run:

```
docker compose up --build
```

Open <http://localhost:3000> and create an account. Check API health at <http://localhost:8000/health>, or explore API calls at <http://localhost:8000/docs>. Stop the app with `Ctrl+C`; use `docker compose down` to remove its containers.

Try it with safe demo data

From the `backend/` directory, run `python -m scripts.seed_demo_data`. It creates six sample applications for `demo@jobtrack.local` with the local password `demo-password`. Running it again replaces only that demo account and its data. Use only local, non-sensitive environments.

Run the checks

```
# In backend/
python -m pip install -r requirements.txt
python -m pytest

# In frontend/
npm ci
npm run lint
npm run typecheck
npm test
npm run build
```

Beginner's learning path

1. Open the app, sign up, and create a sample application.
2. Follow that feature from `Applications.tsx` to the API client and applications router.
3. Read its tests to see valid inputs and edge cases.
4. Make a small change, run focused tests, and observe the result in the browser.

Keep secrets private: `.env` contains passwords and signing secrets. Do not commit it. Use sample information in demos, and never upload a real resume to a public instance.